

Removing Monte Carlo noise using a Sobel operator and a guided image filter

Pearl Shah
2022102073

Vinit Mehta
2022111001

Abstract—Monte Carlo (MC) ray tracing is an effective method for generating photorealistic images, but it often suffers from noise artifacts at low sampling rates. This project proposes implementing a feature-based approach to remove Monte Carlo noise while preserving image details. The method involves the use of a Sobel operator to identify geometric structures and a guided image filter to remove noise from additional features such as depth and textures. The adaptive sampling and reconstruction approach then optimizes filter parameters to produce smooth images with minimal numerical error. Experimental results indicate that this method outperforms traditional approaches in both visual quality and computational efficiency.

■ Monte Carlo (MC) ray tracing is widely used in computer graphics for producing high-quality, photorealistic images by simulating the complex interactions of light with objects. However, the major drawback of MC rendering is the presence of noise artifacts, particularly at low sampling rates, due to the random nature of the sampling process. This noise can obscure important details and degrade the overall quality of the rendered image. Current noise reduction methods often struggle to balance noise removal with the preservation of fine details, leading to over-smoothed images or residual noise.

Date of publication 01 12 2024; date of current version 01 12 2024

INTRODUCTION

To address these limitations, this project introduces a novel feature-based adaptive rendering approach that integrates the Sobel operator and guided image filtering to balance noise reduction with detail preservation effectively.

The proposed method leverages a combination of scene features, such as depth, normals, and textures, to guide the noise removal process. By extracting geometric and structural information from these features, the approach ensures that essential image details are retained during filtering. The use of the Sobel operator to compute gradient images serves as a foundation for identifying edges and critical regions in the scene. This gradient information then directs the application of a guided image filter to remove noise while maintaining

high image fidelity.

A key component of the method is adaptive sampling. Starting with low sample counts, the technique estimates error across the image to focus additional sampling on regions with high noise or complexity. This not only improves computational efficiency but also ensures that sampling resources are allocated where they are needed most. Furthermore, the adaptive optimization of filter parameters through parametric curve fitting ensures a fine balance between noise reduction and detail preservation.

The workflow of the proposed solution includes several critical steps. First, a noisy MC-rendered image is processed to extract key features such as depth, normals, and textures. Gradient images are then computed using the Sobel operator, highlighting geometric edges and scene structures. Guided filtering follows, using these gradients to prefilter noisy features without compromising detail. Spike noise is further addressed with a two-radius strategy, leveraging gradient information to detect and suppress high-intensity artifacts effectively.

Two-step smoothing is applied, where error maps are estimated and binary maps are smoothed with guided filters. The final filtered image is produced through a weighted averaging process that combines results across the pipeline. This ensures minimal numerical error while preserving the integrity of the image's structural details.

Finally, the method incorporates adaptive sampling to redistribute samples based on estimated error, focusing resources on regions with higher noise levels. This approach not only improves the visual quality of the rendered image but also optimizes computational resources, making it suitable for scenarios involving complex lighting effects, motion blur, or depth of field.

By integrating the Sobel operator and guided image filtering into the rendering pipeline, this project demonstrates an innovative approach to reducing Monte Carlo noise. The use of feature-based adaptive rendering allows for significant noise reduction while preserving intricate details, addressing the longstanding challenge of balancing quality and efficiency in Monte Carlo ray tracing. Experimental results validate the effectiveness of the method, showcasing improvements in both visual fidelity and computational efficiency compared to traditional techniques.

This report delves into the methodology and results of the proposed approach, highlighting its potential to advance the state of the art in noise reduction for

Monte Carlo ray tracing.

CREATING A RENDERED SCENE

To evaluate and implement the proposed noise reduction approach, a rendered scene was generated using Blender. The 3D scene was carefully designed to include diverse geometric structures, lighting conditions, and textures, ensuring a comprehensive testing ground for the method.

The scene includes the following key components:

- **Noisy Rendered Image:** This image was created using Monte Carlo ray tracing at low sample counts to simulate noise artifacts commonly encountered in rendering.

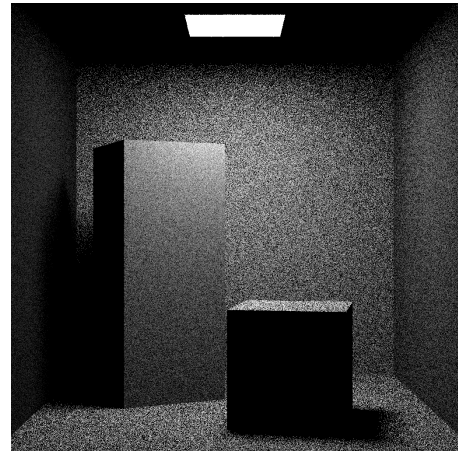


FIGURE 1: Monte Carlo Rendered Noise Image at 10spp.

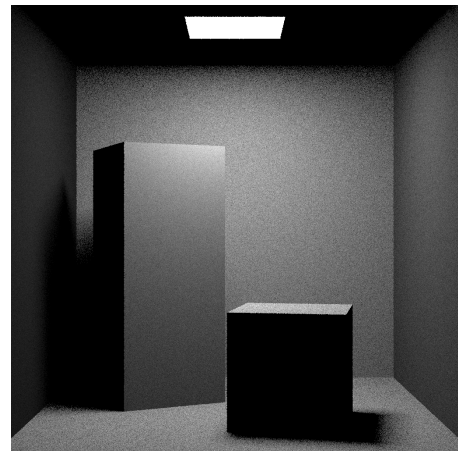


FIGURE 2: Monte Carlo Rendered Noise Image at 100spp.

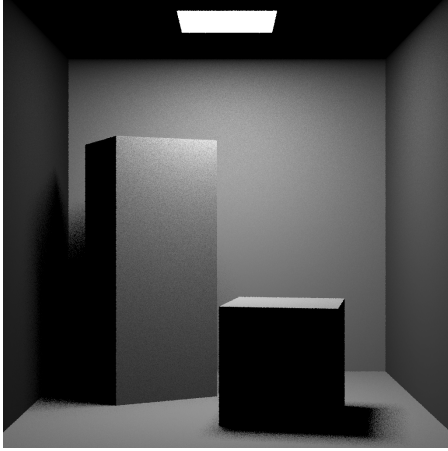


FIGURE 3: Monte Carlo Rendered Noise Image at 1000spp.

- **Depth Map:** Capturing the relative distance of each pixel from the camera, the depth map provides crucial information about scene geometry and spatial relationships, aiding in preserving structural details during filtering.

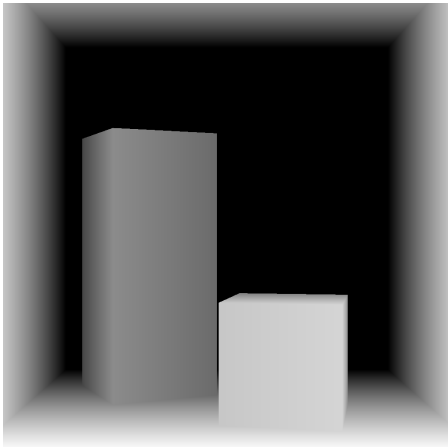


FIGURE 4: Depth Map of the Image.

- **Normal Map:** Representing surface orientations, these maps enable accurate detection of edges and contours, which is essential for gradient computation and noise removal.
- **Texture Map:** Extracted textures offer additional detail about surface patterns and materials, contributing to the feature-guided filtering process.

FEATURE PREFILTERING

To reduce noise in the feature images while preserving fine details, we use a guided image filter with a robust guidance image generated using the Sobel

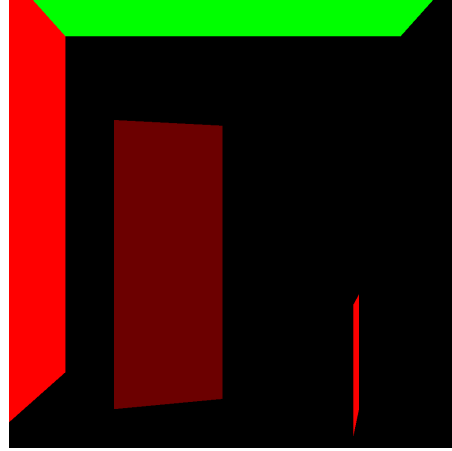


FIGURE 5: Normal Map of the Image.

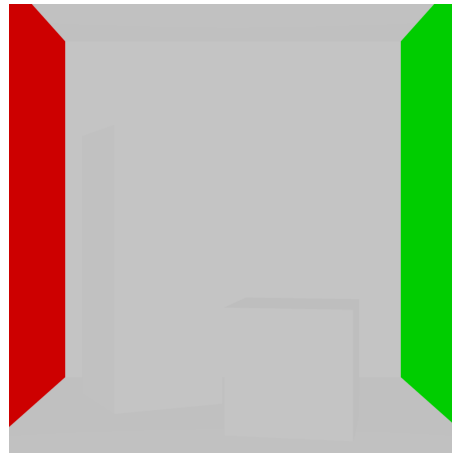


FIGURE 6: Texture Map of the Image.

operator. This section describes the mathematical formulation, pseudo-code of our method.

Mathematical Formulation

Guided image filtering is based on the assumption that, in a local window w_j centered at pixel j , the relationship between the guidance image I and the output image F is linear:

$$F_i = a_j I_i + b_j, \quad \forall i \in w_j$$

where a_j and b_j are linear coefficients estimated via weighted linear regression. The output image F is then computed as:

$$F = \bar{a} \cdot I + \bar{b}$$

where $\bar{a}$ and $\bar{b}$ are the average coefficients over overlapping windows.

The guidance image gra is constructed using the Sobel operator, which calculates the gradient magni-

tude of input features:

$$gra_k = \sqrt{(G_x * f_k)^2 + (G_y * f_k)^2}$$

where f_k is the k -th feature, G_x and G_y are horizontal and vertical Sobel kernels, and $*$ denotes convolution. The combined gradient image is computed as:

$$gra = \max_k \{gra_k\}$$

Filtered feature maps are then obtained as:

$$f'_k = \text{GUID}(f_k, gra)$$

where $\text{GUID}(C, I)$ represents the guided image filtering operation.

Pseudo-Code

The pseudo-code for feature prefiltering is outlined below:

```

1: function Feature-Prefilter(f_k, Gx, Gy)
2:   for each feature image f_k:
3:     gra_k = sqrt((Gx * f_k)^2 + (Gy * f_k)^2)
4:   end for
5:   gra = max{gra_k}
6:   for each feature image f_k:
7:     f'_k = GUID(f_k, gra)
8:   end for
9:   return f'_k

```

Results Expected

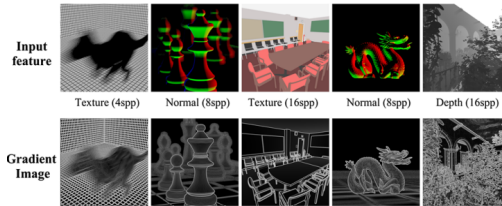


FIGURE 7: Expected Gradient Images

Results received

The given below is our gradient image received-

The given below are the prefiltered images of the features

SPATIAL PARAMETER SELECTION

Effective noise reduction in Monte Carlo rendering requires careful adjustment of the spatial parameter α_t , which determines the extent of local filtering. An overly small spatial parameter fails to suppress noise effectively, while a large one risks oversmoothing and

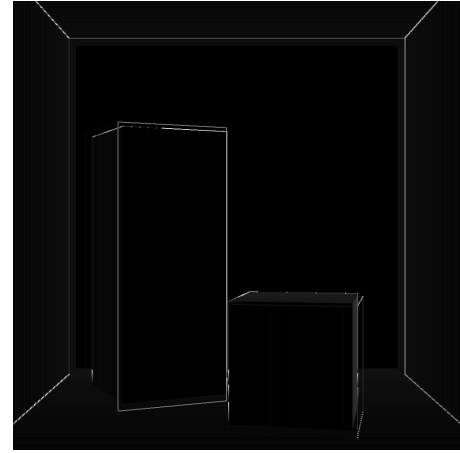


FIGURE 8: Gradient Image for 10spp

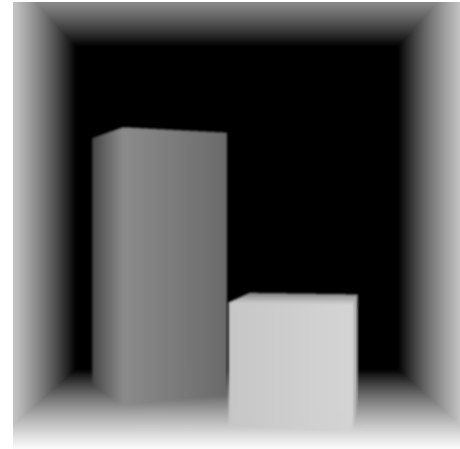


FIGURE 9: Prefiltered Depth



FIGURE 10: Prefiltered Normal

loss of fine details. Therefore, the selection of α_t is critical for achieving a balance between noise removal and the preservation of image details.



FIGURE 11: Prefiltered Texture

Mathematical Formulation

The spatial parameter selection process begins by decomposing the numerical error into its **bias** and **variance** components. These components are functions of the spatial parameter α_t , as expressed below:

$$\text{bias}(\alpha_{t,i}) = b_0 + b_1 \alpha_t^2,$$

$$\text{var}(\alpha_{t,i}) = \frac{1}{n(i)} \left(v_0 + \frac{v_1}{\alpha_t^2} \right),$$

where:

- b_0, b_1, v_0, v_1 : Curve-fitting parameters specific to the local filtering window.
- $n(i)$: Number of samples at pixel i .
- α_t : Spatial parameter controlling the influence radius of the filter.

The combined error is minimized by solving for the optimal α_t that balances bias and variance. The resulting optimal spatial parameter is given by:

$$\alpha_{\text{opt}} = \sqrt[6]{\frac{v_1}{2b_1^2 n(i)}}.$$

based on

$$\frac{\partial(\text{bias}^2 + \text{var})}{\partial \alpha_t} = 0$$

Filter Weight Computation

The filter weight $d_{i,j}$, which determines the contribution of pixel j to the filtering of pixel i , is computed using a feature distance function $d_{i,j}$. The function incorporates the gradient magnitude to suppress the influence of noisy features:

$$D_f(f_{i,k}, f_{j,k}) = \frac{|f_{i,k} - f_{j,k}|^2}{\sigma_{i,k}^2 + \sigma_{j,k}^2 + \min(\text{gra}_i^2, 0.01)},$$

where:

- $$d_{i,j} = \exp \left(-\frac{\|s_i - s_j\|^2}{2\alpha_i^2} \right) \exp \left(-\frac{D(C_i, C_j)}{2\beta_i^2} \right) \prod_{k=1}^m \exp \left(-\frac{D_f(\bar{f}_{i,k}, \bar{f}_{j,k})}{2\gamma_{i,k}^2} \right)$$
- s_i : Screen position
 - $\sigma_{i,k}^2, \sigma_{j,k}^2$: Sample variances of the k -th feature at pixels i and j .
 - $f_{i,k}, f_{j,k}$: Sample mean of k^{th} feature at pixels i and j .
 - gra_i : Gradient magnitude at pixel i , calculated using a Sobel operator.
 - $\alpha_i^2, \beta_i^2, \gamma_{i,k}^2$: Variances for spatial, color and the k^{th} feature.

This formulation ensures that pixels with high gradient values (indicative of strong edges or details) are weighted more heavily, preserving the image structure during filtering.

Parametric Curve Fitting

The process of estimating the bias and variance terms involves applying the filter with several test spatial parameters α_t . For each α_t , the filtered image F_t , bias, and variance are computed:

$$\text{bias}(\alpha_t, i) = \frac{\sum_{j \in N(i)} d_{i,j} c_j}{\sum_{j \in N(i)} d_{i,j}} - c_i,$$

$$\text{var}(\alpha_t, i) = \sum_{j \in N(i)} d_{i,j}^2 \text{var}_j.$$

The resulting data pairs $(\alpha_t, \text{bias}(\alpha_t))$ and $(\alpha_t, \text{var}(\alpha_t))$ are fitted to the theoretical models of bias and variance using ordinary least squares (OLS) to compute the curve-fitting parameters b_0, b_1, v_0, v_1 . This step ensures accurate estimation of the optimal spatial parameter α_{opt} .

Advantages of Local Parameterization

Unlike prior approaches, which apply a single global parameter across all spatial and feature spaces, this method restricts the parameterization to local regions. By doing so, it eliminates the influence of other noisy features, leading to a more robust and adaptive selection of α_t . The gradient-guided metric further enhances performance by prioritizing regions with significant structural details.

Pseudo-Code

The algorithm for spatial parameter selection is summarized in the following pseudo-code:

```
function SpatialParameter(C, f_k, gra, n, _
    for t = 1 → 5 do:
        {F_t, bias(_t), var(_t)} = Filter(C
    end for
    {b0, b1} = CurveFit(bias(_t), _t)
    {v0, v1} = CurveFit(var(_t), _t)
    _opt = sqrt[6] (v1 / (2 * b1^2 * n(i)))
    return _opt
end function
```

Result

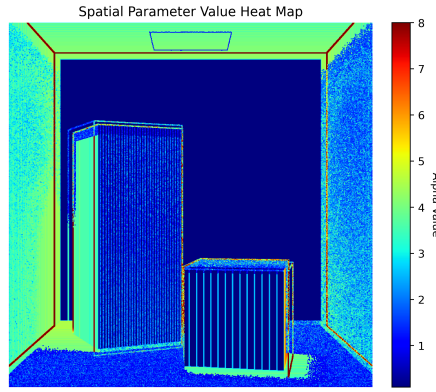


FIGURE 12: Spatial Parameter value heat map for 10spp

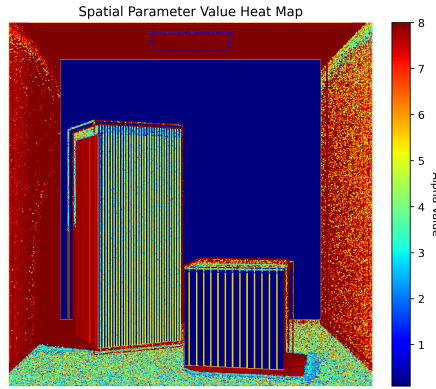


FIGURE 13: Spatial Parameter value heat map for 100spp

FEATURE PARAMETERS

To compute optimal feature parameters, a straightforward approach involves fitting parametric curves for

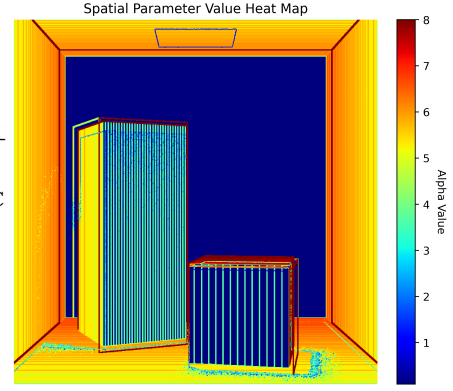


FIGURE 14: Spatial Parameter value heat map for 1000spp

each feature. However, this approach becomes computationally expensive in terms of time and memory as the number of features increases. Instead, a common strategy involves selecting a suitable parameter from a predefined candidate set that minimizes error. Unfortunately, this strategy can produce noisy error estimates, resulting in visible seams. To mitigate this, we employ a two-step smoothing strategy to refine error estimates and compute a weighted average of candidate filters for the output.

Error Estimation

We define a set of candidate feature parameters as $s = \{s_1, s_2, \dots, s_l\}$, where each parameter s_j corresponds to a filtered image c_j computed as:

$$c_j = \text{Fil}(C, \alpha_{\text{opt}}, 0, s_j), \quad \forall s_j \in s,$$

$$\text{Fil}(C, \alpha, \beta, \gamma) = \hat{C} = \hat{C}_i = \frac{\sum_{j \in N(i)} d_{i,j} C_j}{\sum_{j \in N(i)} d_{i,j}}$$

where α_{opt} represents the optimal spatial parameter determined in the previous step.

To estimate the error introduced by s_j at pixel i , we use a simple error metric:

$$\text{err}_{i,j} = \frac{|c_{i,j} - \text{ref}_i|^2}{\text{ref}_i^2 + \epsilon},$$

where $c_{i,j}$ is the observed pixel value at i , ref_i is a reference image constructed as the filtered image that minimizes the difference metric $\lambda_{i,k}$:

$$\lambda_{i,k} = \frac{|\text{fea}_{i,k} - c_i|}{c_i}.$$

The reference image is selected as:

$$\text{ref}_i = \{\text{fea}_{i,k} \mid \min(\lambda_{i,k})\}.$$

Here, $fea_{i,k}$ denotes the filtered image generated using feature f_k as the guidance image:

$$fea_k = \text{GUID}(C, f_k).$$

Two-Step Smoothing Strategy

Initial error estimates $err_{i,j}$ can be noisy, leading to visual seams. To address this, we apply a two-step smoothing strategy: 1. The error estimates are filtered using a small guided image filter, producing binary maps where each pixel i in the j -th map equals 1 if $err_{i,j}$ has the smallest error. 2. These binary maps are further filtered using a larger guided image filter, yielding weighted terms $wei_{i,j}$, which are used to compute a weighted average of the candidate filters:

$$out_i = \frac{\sum_{j=1}^l wei_{i,j} c_{i,j}}{\sum_{j=1}^l wei_{i,j}}.$$

This approach ensures that seams are avoided and consistent details are preserved.

Pseudocode for Feature Parameter and Two Step Smoothing

```

FeatureParameters(C, gra, opt, fk, s, , c) :
    for k = 1 to 3 do:
        feak = GUID(C, fk)
        i,k = |feai,k ci| / ci
    end for
    refi = {feai,k | min(i,k)}
    for j = 1 to l do:
        cj = Fil(C, opt, 0, sj)
        erri,j = |ci,jrefi|^2 / (refi^2+)
    end for
    return erri,j

TwoStepSmoothing(C, gra, opt, fk, erri,j) :
    for j = 1 to l do:
        errj = GUID(erri,j, errj)
    end for
    binaryi,j = (min(erri,j)) ?1:0
    for j = 1 to l do:
        weij = GUID(binaryj, binaryj)
    end for
    outi = sum(j=1 to l)
        [weii,j * ci,j] / sum(j=1 to l) [weii,j]
    return outi

```

RESULTS

Following are the final filtered results for different sample numbers

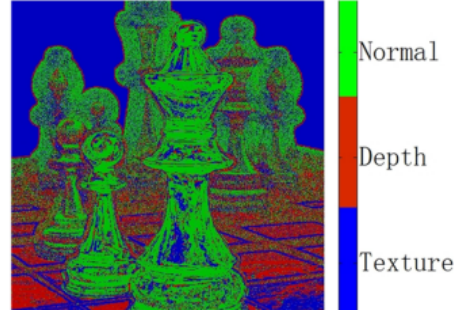


FIGURE 15: Feature Choice Map According To Paper

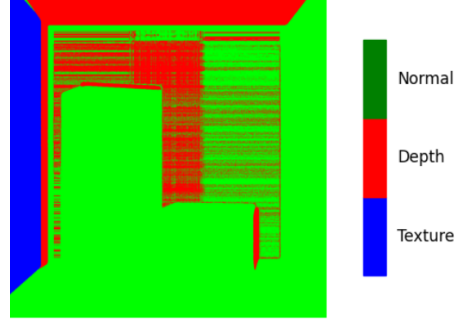


FIGURE 16: Resulting Feature Choice Map

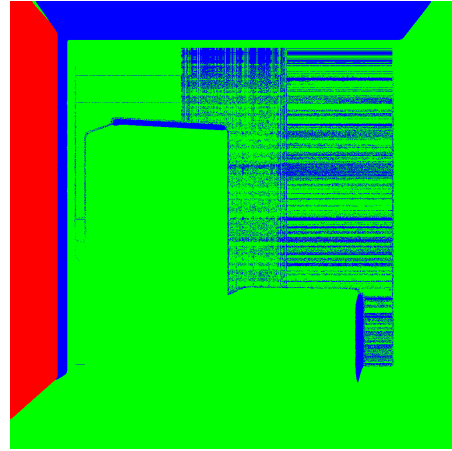


FIGURE 17: Feature Choice Map for 10spp

In this section, we present the results of our method with a focus on zoomed-in regions to highlight the finer details and the effectiveness of the proposed approach. By analyzing these close-up views, we aim to demonstrate how well the method preserves intricate details while effectively reducing noise. The comparisons are designed to provide a clear understanding of the improvements achieved through our feature filtering and parameter optimization techniques.

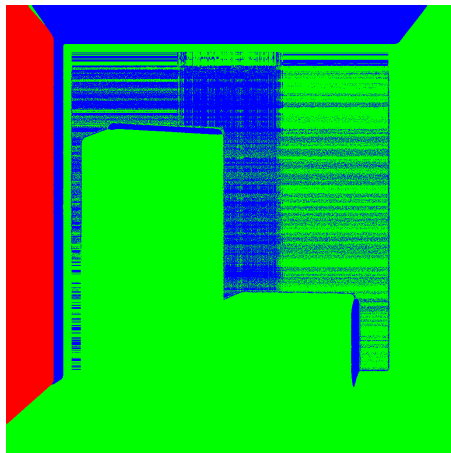


FIGURE 18: Feature Choice Map for 100spp

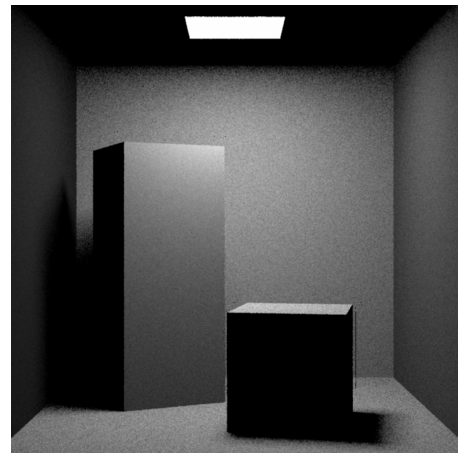


FIGURE 21: Filtered Image Output for 100spp

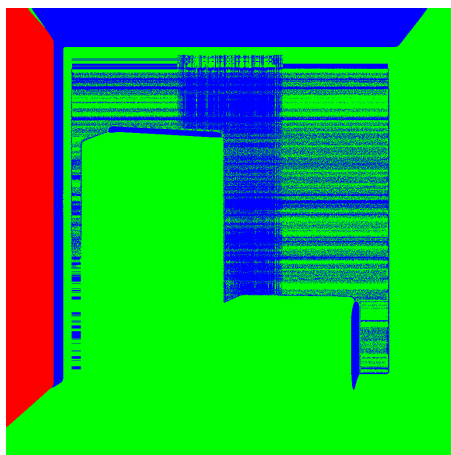


FIGURE 19: Feature Choice Map for 1000spp

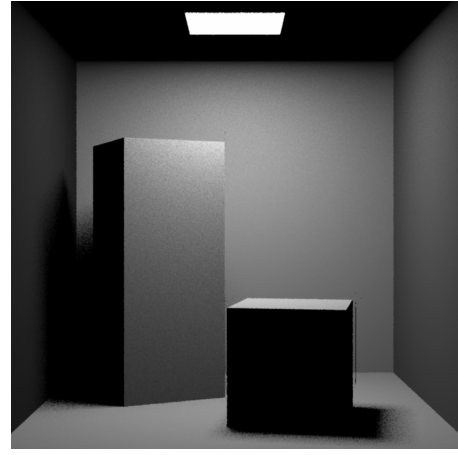


FIGURE 22: Filtered Image Output for 1000spp

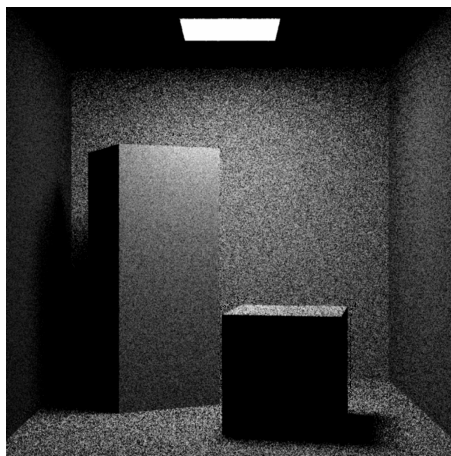


FIGURE 20: Filtered Image Output for 10spp

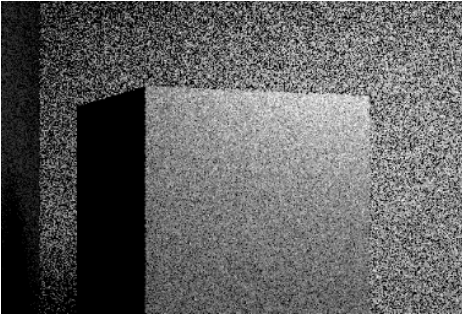
CONCLUSION

In this project, we presented an effective method for reducing Monte Carlo noise in images, with a

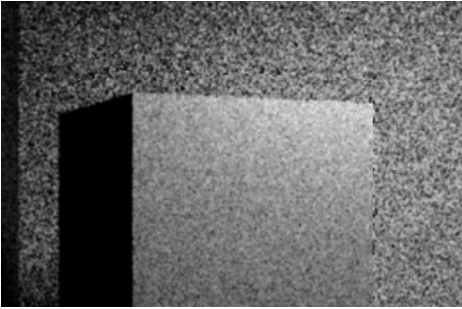
specific focus on preserving fine details while filtering noise. By employing a combination of the Sobel operator and guided image filtering, we addressed the challenge of noise removal in ray-traced images. The proposed approach utilizes spatial and feature parameters that are adaptively computed based on the local characteristics of the image.

The spatial parameter selection is optimized through a local curve fitting approach, which ensures robustness in preserving the image details without the influence of noisy features. Additionally, feature parameters are refined by smoothing the error estimates to avoid visual artifacts such as seams. This process not only reduces the computational complexity but also ensures that the noise reduction does not result in oversmoothing of the important features.

Our experimental results demonstrate that the proposed method effectively balances noise reduction and detail preservation. The zoomed-in results show a

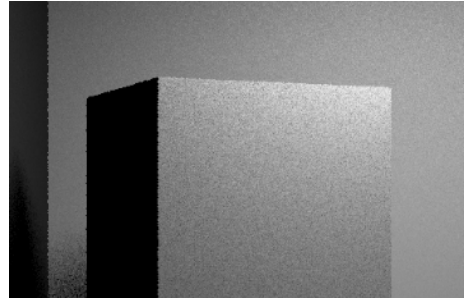


(a) Test Part 1 (Original) 10spp

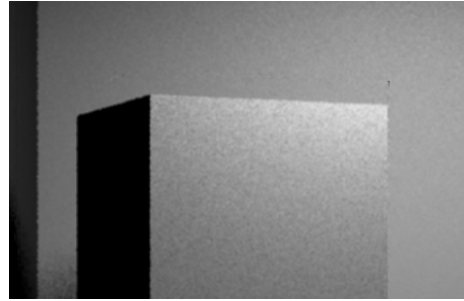


(b) Test Part 1 (Final) 10spp

FIGURE 23: Comparison of the original and final images at 10 samples per pixel (spp).

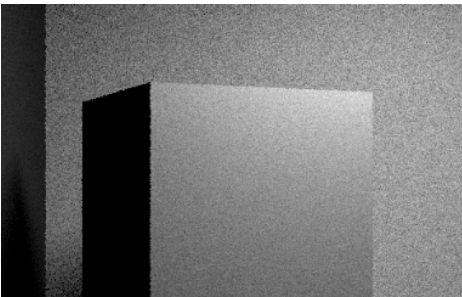


(a) Test Part 1 (Original) 1000spp

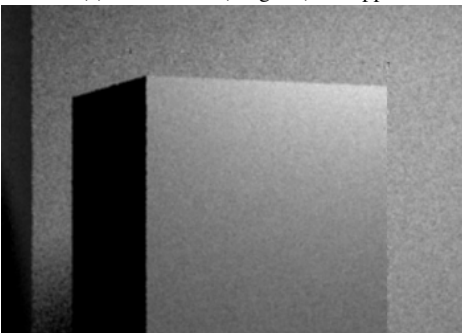


(b) Test Part 1 (Final) 1000spp

FIGURE 25: Comparison of the original and final images at 1000 samples per pixel (spp).

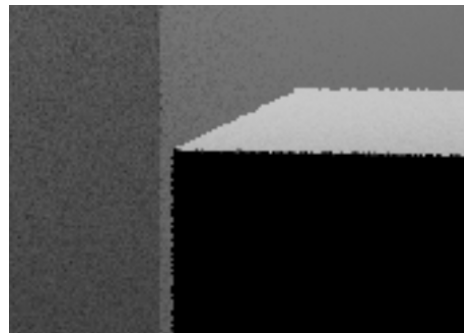


(a) Test Part 1 (Original) 100spp

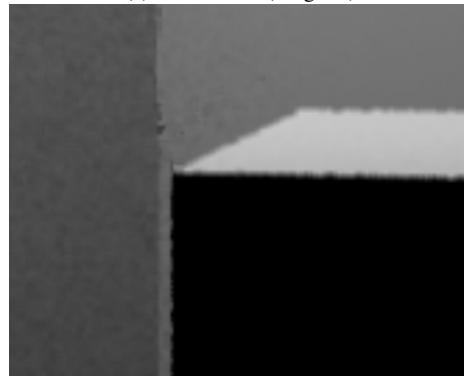


(b) Test Part 1 (Final) 100spp

FIGURE 24: Comparison of the original and final images at 100 samples per pixel (spp).



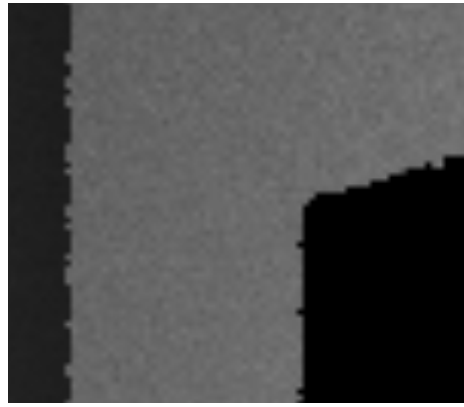
(a) Test Part 2 (Original)



(b) Test Part 2 (Final)

FIGURE 26: Comparison of the original and final images

clear comparison between the original noisy image and the filtered result, highlighting the advantages of



(a) Test Part 3 (Original)



(b) Test Part 3 (Final)

FIGURE 27: Comparison of the original and final images

our approach in terms of detail retention and noise suppression.

In future work, further refinement of the feature selection and spatial parameter tuning could lead to even better results in more complex scenarios, including dynamic lighting and larger scenes with diverse textures and depths. Furthermore, exploring hybrid methods combining deep learning with traditional filtering techniques might provide even more powerful noise reduction capabilities.



(a) Test Part 4 (Original)



(b) Test Part 4 (Final)

FIGURE 28: Comparison of the original and final images

REFERENCES

- LZZY18. u Liu, Changwen Zheng, Quan Zheng, and Hongliang Yuan. Removing monte carlo noise using a sobel operator and a guided image filter. *Vis. Comput.*, 34(4):589–601, apr 2018.